Java Backend Architecture

Interview Series

# Chapter 9.5
# Distributed Cache – Hazelcast & Memcached

50+ Interview Q&A | Code Examples | Architecture Diagrams | Advanced Topics

# Chapter 9.5 – Distributed Cache: Hazelcast & Memcached

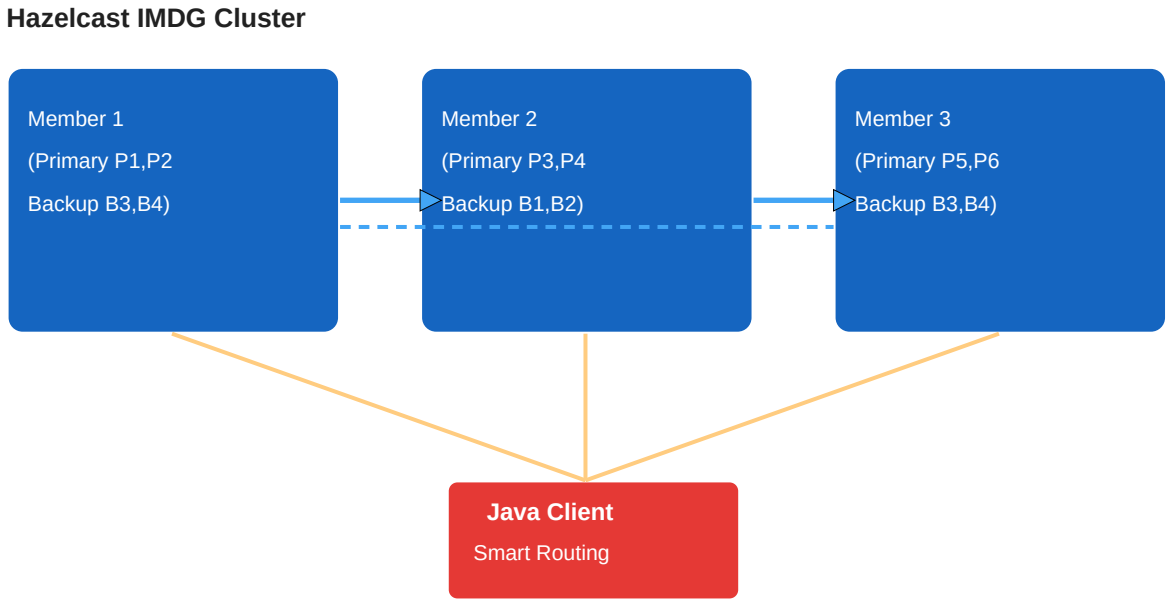
## Introduction

Beyond Redis, Java architects encounter two other distributed caching solutions: **Hazelcast** and **Memcached**. Hazelcast is a Java-native In-Memory Data Grid (IMDG) that runs embedded within the JVM or as a standalone cluster. It distributes data via partitioning with configurable backups and provides a rich distributed computing API (maps, queues, locks, executors, topics). Memcached is a simpler, high-performance, multithreaded key-value cache designed for extremely high throughput with a minimal protocol — no persistence, no replication, just fast in-memory storage using a slab allocator.

## Core Concepts & Definitions

| | |
|---|---|
| **Hazelcast IMDG** | In-Memory Data Grid: distributed, partitioned, replicated data store for Java objects. |
| **IMap** | Hazelcast distributed map; supports listeners, queries, eviction, persistence, near cache. |
| **Partition** | Hazelcast splits keyspace into 271 partitions, distributed across cluster members. |
| **Near Cache** | Local in-process cache for hot Hazelcast entries; reduces network hops; invalidated on update. |
| **MapStore / MapLoader** | Hazelcast SPI: persist IMap entries to DB (write-through/write-behind) or load on miss. |
| **Embedded Mode** | Hazelcast starts inside application JVM; data co-located with compute. |
| **Client-Server Mode** | Hazelcast cluster is separate; Java app uses Hazelcast client with smart routing. |
| **CP Subsystem** | Hazelcast CP mode using Raft consensus; provides linearizable maps, locks, atomic counters. |
| **Memcached** | High-performance, simple key-value cache; multithreaded; no persistence; no replication. |
| **Slab Allocator** | Memcached memory model: pre-allocates fixed-size chunks per slab class; no fragmentation. |
| **LRU (Memcached)** | Per-slab LRU eviction when slab is full; not global LRU across all slabs. |
| **spymemcached / xmemcached** | Java client libraries for Memcached; consistent hashing for multi-node clusters. |
| **Consistent Hashing** | Distribute keys across Memcached nodes; minimize remapping when nodes join/leave. |
| **SASL Auth** | Memcached SASL authentication (binary protocol); not available in ASCII protocol. |
| **Hazelcast Jet** | Stream/batch processing engine built on Hazelcast; distributed DAG-based pipelines. |

## Hazelcast IMDG Cluster Architecture




*Partitions distributed across members; each partition has primary + backup; smart client routes directly*

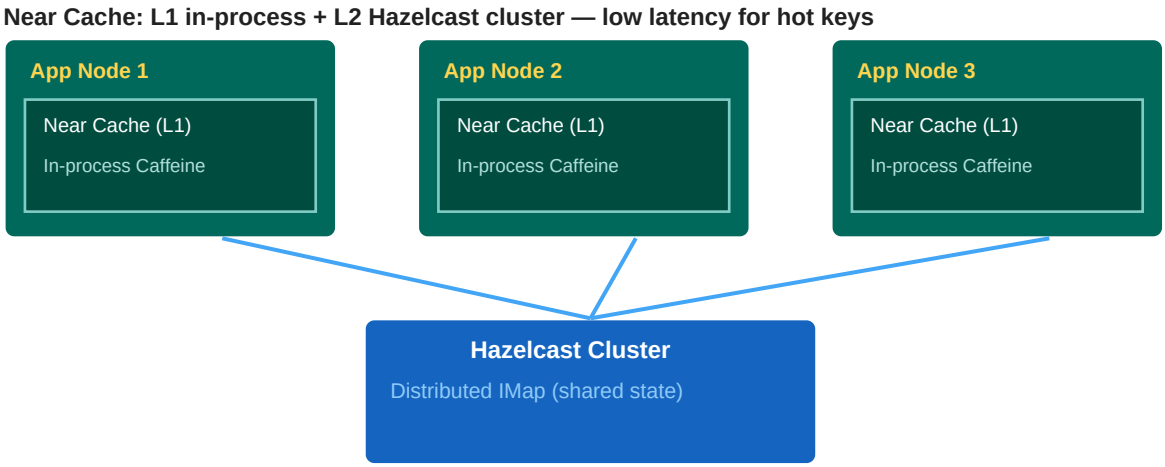
Hazelcast distributes its keyspace across 271 partitions. Each partition has exactly one primary member and configurable backup copies. The smart client knows the partition layout and routes requests directly to the owning member — single hop for most operations. When a member joins or leaves, partitions are rebalanced automatically.

## Hazelcast Setup & IMap Usage

```
<!-- pom.xml --> <dependency> <groupId>com.hazelcast</groupId>
<artifactId>hazelcast</artifactId> <version>5.3.6</version> </dependency>
```

```
// Embedded Mode: Hazelcast runs in the same JVM Config config = new Config();
config.setClusterName("my-cluster"); MapConfig mapConfig = config.getMapConfig("products");
mapConfig.setMaxSizeConfig( new MaxSizeConfig(10_000, MaxSizePolicy.PER_NODE));
mapConfig.setEvictionConfig( new EvictionConfig().setEvictionPolicy(EvictionPolicy.LFU));
mapConfig.setTimeToLiveSeconds(600); // 10 min TTL mapConfig.setMaxIdleSeconds(300); // evict
if idle > 5 min mapConfig.setBackupCount(1); // 1 sync backup HazelcastInstance hz =
Hazelcast.newHazelcastInstance(config); // IMap — distributed Map with all java.util.Map
operations IMap<Long, Product> products = hz.getMap("products"); products.put(42L, product);
// puts in cluster products.put(43L, product2, 5, TimeUnit.MINUTES); // with entry TTL Product
p = products.get(42L); // smart routing products.remove(42L); // removes from cluster // Async
get (non-blocking) CompletionStage<Product> future = products.getAsync(42L);
future.thenAccept(v -> System.out.println("Got: " + v));
```

## Hazelcast Near Cache




*Invalidation: Hazelcast pushes invalidation events to near-cache holders on entry update*

```
// Near Cache config: local L1 cache on client NearCacheConfig nearCacheConfig = new
NearCacheConfig("products") .setInMemoryFormat(InMemoryFormat.OBJECT) // deserialized objects
.setMaxIdleSeconds(60) .setTimeToLiveSeconds(300) .setEvictionConfig( new EvictionConfig()
.setSize(5_000) .setMaxSizePolicy(MaxSizePolicy.ENTRY_COUNT)
.setEvictionPolicy(EvictionPolicy.LFU)) .setInvalidateOnChange(true); // server pushes
invalidations ClientConfig clientConfig = new ClientConfig();
clientConfig.addNearCacheConfig(nearCacheConfig); HazelcastInstance client =
HazelcastClient.newHazelcastClient(clientConfig); IMap<Long, Product> nearMap =
client.getMap("products"); // First get: cluster hop; subsequent gets: from local near cache
Product p1 = nearMap.get(42L); // network Product p2 = nearMap.get(42L); // local near cache —
nanoseconds
```

## Hazelcast MapStore (Read/Write-Through Persistence)

```
@Component public class ProductMapStore implements MapStore<Long, Product> { @Autowired private
ProductRepository repo; @Override public Product load(Long key) { // Called on IMap miss —
read-through return repo.findById(key).orElse(null); } @Override public void store(Long key,
Product value) { // Called on IMap.put — write-through repo.save(value); } @Override public
void delete(Long key) { repo.deleteById(key); } @Override public Map<Long, Product>
loadAll(Collection<Long> keys) { return repo.findAllById(keys).stream()
.collect(Collectors.toMap(Product::getId, Function.identity())); } }
```

```
// Wire MapStore to IMap config MapStoreConfig storeConfig = new MapStoreConfig()
.setImplementation(productMapStore) .setEnabled(true) .setWriteDelaySeconds(0) // 0 =
write-through; >0 = write-behind .setWriteBatchSize(200); // for write-behind batch flush
mapConfig.setMapStoreConfig(storeConfig);
```

## Hazelcast CP Subsystem (Linearizable Distributed Primitives)

```
// CP Subsystem: Raft-based, linearizable (not AP like IMap) CPSubsystem cp =
hz.getCPSubsystem(); // Distributed Lock (FencedLock — linearizable) FencedLock lock =
cp.getLock("my-lock"); long fence = lock.lockAndGetFence(); // returns monotonic fence token
try { // safe critical section with fencing token for external ops doWork(fence); } finally {
lock.unlock(); } // AtomicLong IAtomicLong counter = cp.getAtomicLong("request-counter"); long
val = counter.incrementAndGet(); // Counting Semaphore ISemaphore semaphore =
cp.getSemaphore("db-connections"); semaphore.init(10); semaphore.acquire(); try {
dbOperation(); } finally { semaphore.release(); }
```

## Spring Integration with Hazelcast

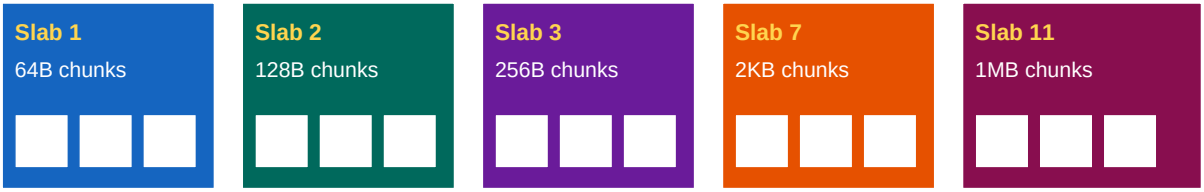
```
@Configuration @EnableCaching public class HazelcastCacheConfig { @Bean public
HazelcastInstance hazelcastInstance() { Config config = new
ClasspathXmlConfig("hazelcast.xml"); return Hazelcast.newHazelcastInstance(config); } @Bean
public CacheManager cacheManager(HazelcastInstance hz) { return new HazelcastCacheManager(hz);
} }
```

### Now @Cacheable / @CacheEvict work with the Hazelcast backend:

```
@Cacheable(value = "products", key = "#id") public Product findById(Long id) { return
repo.findById(id).orElseThrow(); } @CacheEvict(value = "products", key = "#product.id") public
Product update(Product product) { return repo.save(product); }
```

## Memcached Architecture & Slab Allocator

**Memcached Slab Allocator**




Key Assignment: Value fits into smallest slab that can contain it

LRU per slab class — eviction is within slab, not global

Memory is pre-allocated per slab; no fragmentation but internal waste possible

*Slab Reassignment (1.4.x+): slabs can be moved between classes if skewed allocation*

Memcached uses a **slab allocator** to manage memory without malloc/free overhead. Memory is divided into slab classes (exponentially growing chunk sizes). Each value is stored in the smallest slab class that fits it. Within each slab class, eviction uses LRU. This design avoids memory fragmentation at the cost of potential internal waste when values do not fill their chunk exactly.

## Memcached Java Client (spymemcached)

```
<!-- pom.xml --> <dependency> <groupId>net.spy</groupId> <artifactId>spymemcached</artifactId>
<version>2.12.3</version> </dependency>
```

```
// Connect to Memcached cluster MemcachedClient mc = new MemcachedClient( new
ConnectionFactoryBuilder() .setProtocol(ConnectionFactoryBuilder.Protocol.BINARY)
.setHashAlg(DefaultHashAlgorithm.KETAMA_HASH) // consistent hashing .build(),
AddrUtil.getAddresses("mc1:11211 mc2:11211 mc3:11211")); // Set with TTL (seconds)
mc.set("product:42", 600, product); // async, returns Future // Synchronous get Product p =
(Product) mc.get("product:42"); // Atomic add (fails if key exists — prevents double-set)
mc.add("product:42", 600, product); // Atomic CAS (check-and-set for optimistic locking)
CASValue<Object> cas = mc.gets("product:42"); if (cas != null) { CASResponse resp =
mc.cas("product:42", cas.getCas(), 600, updatedProduct); // CASResponse.OK, EXISTS, NOT_FOUND
} // Increment/decrement atomic counter long val = mc.incr("counter:views:42", 1, 0, 600); //
incr by 1, default 0, TTL 600s // Delete mc.delete("product:42"); // Shutdown mc.shutdown();
```

## Memcached vs Redis vs Hazelcast Comparison

| Feature | Memcached | Redis | Hazelcast |
|---|---|---|---|
| Data structures | String only | String, Hash, List, Set, ZSet, Stream | Native objects, Map, Queue, Topic, Set |
| Persistence | None | RDB + AOF | Hot Restart (EE), MapStore |
| Replication | None (client sharding) | Async replica + Sentinel/Cluster | Sync + async backup |
| Threading | Multi-threaded | Single-threaded cmd + I/O threads | Multi-threaded |
| Cluster | Client-side consistent hash | Redis Cluster (16384 slots) | Built-in partitioning (271 parts) |
| Distributed Locks | None | SETNX / Redisson | CP Subsystem FencedLock |
| Near Cache | No | No (client-side with Lettuce tracking) | Yes (built-in) |
| Java POJO caching | Needs serialization | Needs serialization | Native (BINARY/OBJECT) |
| Max value size | 1 MB | 512 MB | No hard limit |
| Pub/Sub | No | Yes | Yes (ITopic) |

## Interview Questions & Answers

### Q1. What is Hazelcast IMDG?

Hazelcast In-Memory Data Grid is a distributed, partitioned, replicated Java data store. It runs embedded in the JVM or as a standalone cluster, providing distributed Maps, Queues, Topics, Locks, and a CP Subsystem (Raft) for linearizable primitives. It integrates with Spring via HazelcastCacheManager.

### Q2. How does Hazelcast partition data?

Hazelcast divides the key space into 271 partitions using consistent hashing (murmur3 on serialized key). Each partition has one primary owner and backup copies on other members. Smart routing clients direct requests to the partition owner in one hop.

### Q3. What is Hazelcast Near Cache?

Near Cache is an in-process L1 cache on the client/member side for hot IMap entries. Reduces cluster hops for frequently read keys. Invalidation is server-pushed: when the IMap entry changes, Hazelcast sends an invalidation event to all near-cache holders. Configured per map name.

### Q4. What is Hazelcast MapStore?

MapStore is an SPI to connect an IMap to a backing store (DB). load() is called on cache miss (read-through). store() is called on put (write-through when writeDelay=0, write-behind when writeDelay>0). Enables Hazelcast as a read/write-through cache tier.

### Q5. What is the Hazelcast CP Subsystem?

A Raft-based strongly consistent (CP) layer within Hazelcast. Provides linearizable FencedLock, IAtomicLong, ISemaphore, CountDownLatch, CPMap. Separate from default AP IMap. Requires dedicated CP member group (minimum 3). Used for distributed coordination, not data caching.

### Q6. What is a FencedLock and why is it safer than SETNX?

FencedLock returns a monotonically increasing fence token on each lock acquisition. The token is passed to external storage operations. Even if GC pause causes delayed unlock, the storage can reject stale fence tokens — prevents split-brain write conflicts. SETNX has no fencing concept.

### Q7. How does Hazelcast handle node failure?

When a member fails, Hazelcast re-assigns its primary partitions to other members. Backup copies are promoted to primary. Rebalancing starts immediately. During rebalancing, some reads may fall back to secondary members. Data is not lost if backup count ≥ 1.

### Q8. What is the difference between Hazelcast embedded and client-server mode?

Embedded: Hazelcast runs inside the application JVM; zero network latency for data operations; all JVMs form the cluster. Client-server: separate Hazelcast cluster; application uses thin client; application and data lifecycle are decoupled; better for microservices and container deployments.

### Q9. What is Memcached and what are its key characteristics?

Memcached is a high-performance, multithreaded, in-memory key-value cache with no persistence, no replication, and no rich data structures. It uses a slab allocator to avoid fragmentation. Designed for extreme read throughput with a simple ASCII or binary protocol. Use for simple string caching when Redis's features are not needed.

### Q10. Explain Memcached's slab allocator.

Memory is pre-allocated per slab class (exponentially growing chunk sizes, e.g. 64B, 128B, 256B). Each value is stored in the smallest fitting slab. LRU eviction operates per slab class. No global memory compaction needed → no fragmentation. Trade-off: internal waste if values do not fill their chunk.

### Q11. How do multiple Memcached nodes work together?

Memcached itself has no clustering — clients shard keys across nodes using consistent hashing (Ketama algorithm). If a node fails, its keys are simply lost (no replication). The client auto-routes to remaining nodes. This simplicity makes Memcached easier to operate but less durable.

### Q12. What is CAS (Check-And-Set) in Memcached?

gets command returns a CAS token with the value. cas command sets the value only if the CAS token matches the server's current token. If another client modified the key between gets and cas, the cas fails (EXISTS). Provides optimistic concurrency control.

### Q13. When would you choose Memcached over Redis?

Memcached excels when: data is simple string/blob, maximum single-node throughput is needed (multi-threaded), no persistence required, low operational complexity desired. Redis is preferred for: complex data types, persistence, pub/sub, Lua scripts, cluster auto-sharding, distributed locks.

### Q14. What is consistent hashing and why is it used in Memcached?

Consistent hashing places nodes on a ring; each key maps to the nearest clockwise node. Adding/removing a node remaps only 1/N of keys (N=number of nodes) rather than all keys. The Ketama algorithm is the standard implementation for Memcached. Without consistent hashing, adding a node would invalidate most of the cache.

### Q15. How does Hazelcast handle Java serialization for IMap values?

Hazelcast supports multiple serialization formats: Java Serializable (default, not recommended), Hazelcast Serializable (IdentifiedDataSerializable — faster, versioned), Portable (queryable without server-side classes), custom Serializer via SerializationConfig. InMemoryFormat.OBJECT stores deserialized Java objects on server — requires server to have the classes.

### Q16. What are the trade-offs of Hazelcast embedded mode vs client-server?

Embedded: zero network latency for data ops, data co-located with compute, but: JVM heap pressure from data + app, cluster includes application nodes (harder to scale independently), deployments restart cluster. Client-server: clean separation, independent scaling of data tier, but adds network RTT (~1ms).

### Q17. What is Hazelcast's write-behind feature?

When MapStore writeDelay > 0, IMap puts are written to the in-memory map immediately and asynchronously flushed to the MapStore in batches after the delay. Improves write throughput. Risk: data loss if member crashes before flush. Mitigate with backup count and Redis-like persistence.

### Q18. What is Hazelcast's Jet product?

Hazelcast Jet is a distributed stream and batch processing engine built on top of Hazelcast's IMDG. Processes DAG pipelines over Kafka, files, IMap sources. Suitable for stateful stream processing co-located with cache data. Competes with Apache Flink for embedded-JVM streaming scenarios.

### Q19. How do you evict entries in Hazelcast IMap?

MapConfig: setTimeToLiveSeconds (since creation), setMaxIdleSeconds (since last access). MaxSizeConfig: entry count, heap percentage, or free heap percentage per node or cluster. EvictionConfig: LRU, LFU, or RANDOM eviction policy. Eviction is per-member, not cluster-global.

### Q20. What is Hazelcast's split-brain protection?

When a network partition occurs, both sides may continue as separate clusters. On healing, Hazelcast performs split-brain merge with configurable MergePolicy (e.g., PutIfAbsentMergePolicy, PassThroughMergePolicy, LatestUpdateMergePolicy). CP Subsystem with Raft consensus prevents split-brain for linearizable operations.

### Q21. How does Memcached handle large values (>1MB)?

Memcached's maximum value size is 1MB by default. Options: increase -I flag (e.g., -I 2m). Better: decompose the large value into chunks and store separately with a manifest key. Or use Redis (up to 512MB). Alternatively, store large binary in object storage and cache only metadata.

### Q22. What are Hazelcast predicates and SQL queries?

IMap.values(Predicate) filters entries: e.g., Predicates.and(Predicates.equal('category','electronics'), Predicates.greaterThan('price', 100)). Hazelcast also supports distributed SQL via HazelcastSqlService. Indexes improve predicate query performance: addIndex(IndexType.HASH, 'category').

### Q23. What is the Memcached binary protocol vs ASCII protocol?

ASCII protocol: simple text commands (set, get, delete), human-readable, but slower parsing. Binary protocol: fixed-header format, faster parsing, supports SASL authentication, quieter bulk commands (getq, setq), vbucket-based routing. Use binary protocol with spymemcached for production.

### Q24. How does Hazelcast discovery work in Kubernetes?

Hazelcast provides hazelcast-kubernetes plugin. Members discover each other via Kubernetes API (list Pod IPs) or DNS-based service discovery. Configure: discovery-strategies with kubernetes plugin. Kubernetes service account needs read permission on pods/endpoints. Also supports Helm chart deployment.

### Q25. What is the xmemcached Java client?

xmemcached is an asynchronous, NIO-based Memcached client for Java with better throughput than spymemcached for high-concurrency scenarios. Supports consistent hashing, binary protocol, SASL auth, weighted server pools, and partition-mode distribution.

### Q26. How would you implement a distributed counter with Hazelcast vs Redis?

Hazelcast: IAtomicLong counter = cp.getAtomicLong('counter'); counter.incrementAndGet(); (CP Subsystem, linearizable, quorum-based, survives failures). Redis: INCR key (single-threaded, atomic on single node; use CLUSTER INCRBY for cluster). Hazelcast CP is preferred for strict consistency in multi-JVM environments.

### Q27. What are Hazelcast EntryListeners?

EntryListener receives ADDED, REMOVED, UPDATED, EVICTED, EXPIRED, MERGED events from IMap. Registered via map.addEntryListener(listener, includeValue). In client-server mode, events delivered to all registered clients. Useful for cache invalidation cascade, audit logging, reactive pipelines.

### Q28. How is memory managed in Hazelcast vs Memcached?

Hazelcast (default): entries stored in JVM heap as Java objects or serialized byte arrays. Subject to GC pressure. Hazelcast HD Memory (EE): off-heap using sun.misc.Unsafe for GC-free operation. Memcached: native C heap with slab allocator; no JVM overhead; more predictable memory usage.

### Q29. What is the Hazelcast WAN Replication feature?

WAN Replication asynchronously replicates IMap changes to a remote Hazelcast cluster (different DC). Used for geo-distributed active-passive or active-active caches. Conflict resolution via merge policy. Enterprise feature. Active-Active supports bi-directional replication with CRDT semantics.

### Q30. How do you benchmark Memcached vs Redis latency?

Use memtier_benchmark (supports both). Typical results on same hardware: Memcached p99 GET ~0.3ms (multi-threaded), Redis p99 GET ~0.5ms (single-threaded command processing). For pure simple string caching at very high QPS, Memcached can outperform Redis due to multi-threading. Redis wins for complex operations and features.

### Q31. Describe a hybrid Hazelcast + Redis caching architecture.

Hazelcast (L1): embedded IMDG stores Java objects directly (no serialization overhead), near cache for hot entries. Redis (L2): persistent distributed cache for cross-cluster sharing. On Hazelcast eviction, listener populates Redis. On Hazelcast miss, check Redis, deserialize into Hazelcast. Provides ultra-low latency from Hazelcast with Redis as durable fallback.

### Q32. What is Hazelcast's Portable serialization?

Portable is a Hazelcast serialization format that allows server-side queries on fields without server having the Java class. Fields are named and typed in a ClassDefinition. Useful for querying IMap on servers that should not have business domain classes. Versioning supported for schema evolution.

### Q33. What happens in Hazelcast when a partition primary and all its backups fail?

Data for those partitions is permanently lost (no backups survive). Hazelcast will log partition loss. Application receives PartitionMigratingException or stale reads. For critical data, use backup-count=2 (sync) + async-backup-count=1, or use MapStore to reload from DB on loss.

### Q34. What is the Hazelcast Quorum (split-brain protection policy)?

QuorumConfig defines minimum cluster size for IMap/IQueue operations. If live members < quorum, reads/writes are rejected (QuorumException). Prevents split-brain writes on minority partition. E.g., QuorumType.READ_WRITE, minimumClusterSize=3 in a 5-node cluster.

### Q35. How does Memcached handle key expiration?

Memcached uses lazy expiration: expired items are not actively deleted, just ignored on access. The slot is reused when a new item needs it. This means expired items can linger until their slab slot is reclaimed. FLUSH_ALL invalidates all keys by advancing the global epoch, not by deleting.

### Q36. What is Hazelcast's IQueue and how is it used?

IQueue is a distributed, persistent (with MapStore) blocking queue. FIFO ordering within a single partition. offer()/poll() are atomic. Used for work distribution across cluster members. Unlike Kafka/RabbitMQ, not designed for high-throughput streaming but for lightweight task queues.

### Q37. Compare Hazelcast vs Apache Ignite.

Both are Java IMDGs. Hazelcast: simpler setup, stronger Spring integration, CP Subsystem for linearizable ops, Jet for streaming. Apache Ignite: stronger SQL support (distributed ANSI SQL, ACID transactions), collocated compute, ML library, supports persistence as primary DB (not just cache). Choose Ignite for SQL-first; Hazelcast for cache-first.

### Q38. How do you configure Hazelcast cluster discovery in a cloud environment?

AWS: hazelcast-aws plugin — discovers members via EC2 API (security group, VPC, tag filters). GCP: hazelcast-gcp plugin — discovers via GCP API. Azure: hazelcast-azure plugin. Multicast (not recommended in cloud — usually blocked). TCP/IP: explicit member list for small static clusters. Kubernetes: hazelcast-kubernetes plugin (most common for container deployments).

### Q39. What is Hazelcast's Hot Restart feature?

Hot Restart (Enterprise) persists Hazelcast data to disk (NVMe) and restores on restart. Avoids cluster warm-up time. Uses persistent memory format (not Java serialization). Recovery is fast — GB/s from NVMe. Similar to Redis RDB but designed for larger datasets. Cluster can restart without cache miss storm on backing DB.

### Q40. What serialization format gives the best Hazelcast performance?

IdentifiedDataSerializable: fastest Hazelcast-native format; no reflection; explicit readData/writeData methods; factory-based deserialization. For off-heap (HD Memory): requires serialized bytes anyway. Kryo or FST via custom Serializer are also fast for large payloads.

### Q41. How do you monitor Hazelcast in production?

Management Center UI: live cluster view, map stats, partition distribution, member logs. JMX: expose Hazelcast MBeans via hazelcast.jmx=true. Metrics: Hazelcast 4+ has Micrometer support (hazelcast.metrics.enabled=true). Key metrics: map.get/put/remove rates, near-cache hitRate, partition migration count, CP leader elections.

### Q42. What is the Hazelcast Tiered Storage feature?

Hazelcast 5.3 Tiered Storage: hot data in memory (heap or HD Memory), cold data spilled to NVMe SSD. Background promotion/demotion based on access frequency. Enables datasets larger than RAM with near-memory latency for hot tier. Enterprise feature.